

APUNTESWEB · INGENIERÍA INFORMÁTICA

COMPINT

Tercer curso · 2.º cuatrimestre · 5 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción a la Compilación

El problema de la traducción de código no es nuevo, surge desde la programación del primer ordenador.

1.1 Lenguajes de Programación

Un lenguaje de programación es la herramienta mediante la cual los humanos dan instrucciones a las máquinas. Según su nivel de abstracción:

- **Lenguajes Máquina:** son **específicos de cada ordenador**. Consisten en secuencias de unos y ceros. Son inteligibles para el humano promedio pero nativos para el hardware
- **Lenguajes Ensamblador:** también **específicos de cada ordenador**, pero un paso más arriba. Proporcionan **nombres simbólicos** para:
 - **Instrucciones sencillas**
 - **Posiciones de memoria**
- **Lenguajes de Alto Nivel:** Son **independientes de la máquina**. Permiten abstracciones complejas como estructuras de control, variables con tipo, procedimientos, recursividad y tipos abstractos de datos.
- **Lenguajes Orientados a Problemas:** diseñado para dominios específicos. Su objetivo es reducir el tiempo de programación, mantenimiento y depuración.

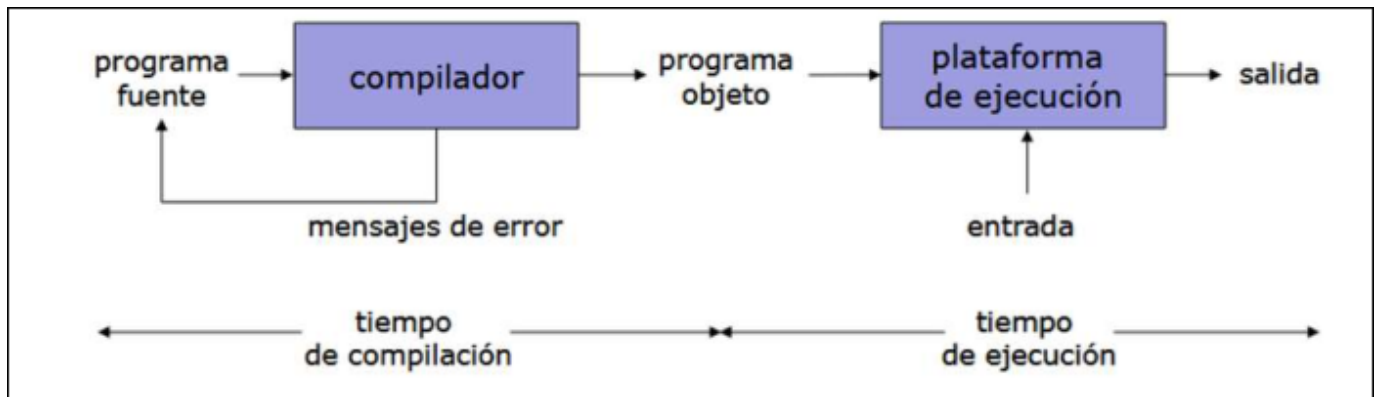
1.2 Procesadores de Lenguajes

Un procesador de lenguaje es un **software encargado de procesar o traducir un programa fuente**.

1.2.1 El Compilador

Es un programa que **traduce un programa** escrito en un **lenguaje fuente** a un programa equivalente escrito en un **lenguaje objeto**.

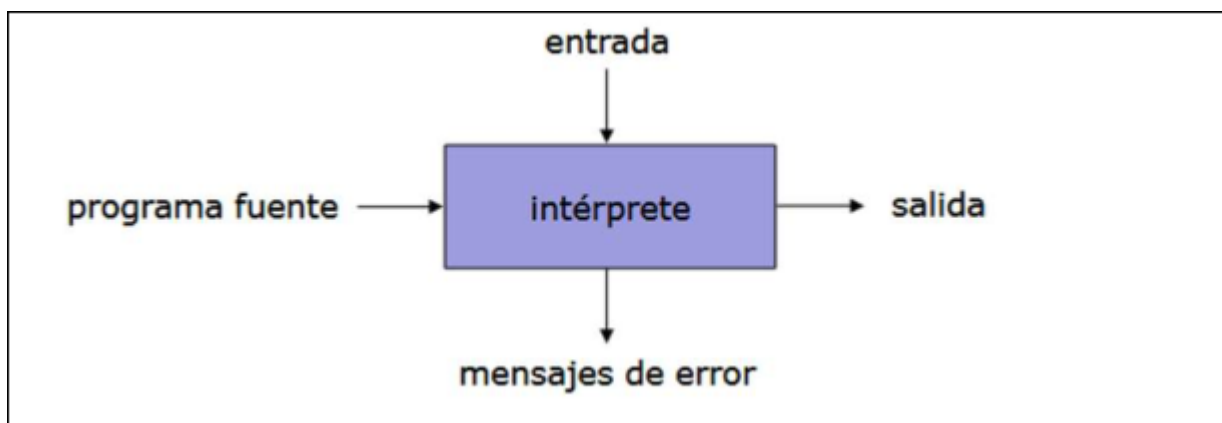
- **Genera un programa objeto** (generalmente es **lenguaje máquina**). El lenguaje objeto es un lenguaje máquina.
- **Informa de la presencia de errores en todo el código fuente.**



- Se compila una vez, se ejecuta n veces. - La ejecución es ****rápida**** - La ****gestión de errores abarca todo el programa**** - Mejor adaptados a entornos de ****producción**** - *Ejemplos:* FORTRAN, C, PASCAL ...

1.2.2 El Intérprete

A diferencia del compilador, el intérprete **no produce un programa objeto. Aparenta ejecutar directamente** cada instrucción del programa fuente utilizando las **entradas** proporcionadas por el usuario. **Traduce y ejecuta instrucción por instrucción.**



- Se ****traduce cada vez**** que se ejecuta - Necesita ****menos memoria**** - Se permite la ****interacción**** con el código en ****tiempo de ejecución**** - Mejor adaptados a entornos de ****desarrollo y experimentación**** - *Ejemplos:* BASIC, LISP, PROLOG ...

1.2.3 Enfoques Híbridos y Modernos

La distinción estricta entre compilador e intérprete se desdibuja en los lenguajes modernos para aprovechar lo mejor de ambos mundos.

Compilador - Intérprete

El proceso se divide en dos fases:

- **Compilación a Lenguaje Intermedio:** El código fuente se compila a un formato intermedio, no a código máquina real.

- **Interpretación (Máquina Virtual):** Una máquina virtual interpreta ese código intermedio en el ordenador destino.
- *Ejemplo: Java.* El compilador genera **bytecode**, que luego es interpretado por la JVM (Java Virtual Machine).

Compiladores JIT (Just-In-Time)

Para mejorar la eficiencia de los sistemas basados en máquinas virtuales o intérpretes, se utilizan los compiladores JIT.

- **Funcionamiento:** compilan en **tiempo de ejecución fragmentos del código intermedio** directamente a código objeto.
- **Ventaja:** mejora la **eficiencia de los intérpretes** y en ocasiones la de los propios compiladores.
- *Ejemplos:* JAVA, PHP, PYTHON ...

1.3 El proceso de traducción

Esta imagen muestra una **cadena clásica de traducción** de un programa, típica de lenguajes como C/C++. Algunos compiladores pueden generar el código máquina directamente.

Partimos de:

```
#include <stdio.h>

int main() {
    printf("Hola\n");
    return 0;
}
```

Este archivo todavía no se puede compilar porque tiene cosas como `#include <stdio.h>` por ello pasa al **preprocesador**. Después pasa al compilador que lo traduce a ensamblador

```
mov eax, 0
call printf
ret
```

El compilador se centra en traducir optimizar el programa. Posteriormente el **ensamblador** convierte ese ensamblador en **código máquina relocable**. Que sea relocable significa que todavía no se sabe exactamente en qué posición de memoria estará cada cosa cuando el programa final se ejecute. Por ejemplo si el programa llama a :

```
printf("Hola\n");
```

El código usa `printf` pero `printf` no está descrito dentro del archivo, está en una biblioteca externa. Entonces el ensamblador puede decir, aquí hay una llamada a `printf` pero todavía no sé la dirección exacta de `printf`. Por eso genera un **archivo objeto**, por ejemplo: `main.o`.

Y por último el **enlazodr** junta el código con otros códigos necesarios:

```
main.o
+ biblioteca estándar de C
+ código de arranque del programa
+ otras bibliotecas
= ejecutable final
```

NOTA

El **lenguaje ensamblador** es una forma textual y más legible de escribir instrucciones de máquina. El **ensamblador** también es un programa que traduce lenguaje ensamblador a código máquina. En Linux, el ensamblador típico se llama `as`.

1.4 Estructura Interna de un Compilador

La siguiente imagen es una ampliación de la fase **compilador** de la imagen anterior.

1.4.1 Estrategia para diseñar compiladores

Si tenemos M lenguajes fuente y N plataformas, crear un compilador para cada combinación requeriría $M \times N$ compiladores. La solución es usar un **Código Intermedio**.

El código intermedio es una representación del programa que no es exactamente el lenguaje original, pero tampoco es todavía código máquina de un ordenador concreto. Si partimos de:

```
int x = 2 + 3;
int y = x * 4;
```

Se convertiría por ejemplo en:

```
t1 = 2 + 3
x = t1
```

```
t2 = x * 4
y = t2
```

No es C pero tampoco es ensamblador x86, ARM o RISC-V. Esto sirve como un **punto universal** entre muchos lenguajes y muchas máquinas. Sin esto si tienes M lenguajes fuente y N plataformas. Sin código intermedio necesitaría un compilador específico para cada combinación. $M \times N$ compiladores. Con código intermedio con $M + N$ partes apañamos.

Plataforma: destino donde se va a ejecutar el programa.

1.4.2 Analizador Léxico

Lee el **flujo de caracteres** del programa fuente y los agrupa en **componentes léxicos**. Cada componente léxico produce como salida un **token** de la forma `<nombre, valor-atributo>`, que pasa a la fase siguiente.

- *Ejemplo:* Identifica `posicion`, el símbolo `=`, el identificador `inicial`, etc.
- *Salida:* Un flujo de tokens.

1.4.3 Analizador Sintáctico

Recibe los tokens y crea una **representación intermedia jerárquica** en forma del **árbol** que describe la **estructura gramatical** del **flujo de tokens**.

- **Árbol sintáctico:** cada **nodo interior** es una **operación** y los **hijos** son los **argumentos**.
- *Ejemplo:* Crea un árbol donde `*` (multiplicación) es hijo de `+` (suma), respetando la precedencia matemática.

1.4.4 Analizador Semántico

Revisa el **árbol sintáctico** y la **tabla de símbolos** para comprobar la **consistencia semántica**. Su tarea principal es la **verificación de tipos**.

- **Coerción:** si el lenguaje lo permite, el analizador puede convertir tipos automáticamente.
- *Ejemplo:* En `velocidad * 60`, si `velocidad` es un número real y `60` es entero, el analizador convierte el `60` a real (`entareal(60)`) para que la operación sea válida.

1.4.5 Generador de Código Intermedio

Traduce el árbol a un código para una máquina abstracta. Debe ser fácil de **producir y traducir a lenguaje objeto**. Una representación muy común del código intermedio es el **código de 3 direcciones**: una secuencia de instrucciones que tienen como máximo 3 operandos.

```
temp1 = entareal(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```

1.4.6 Optimizador de Código

Intenta mejorar el **código intermedio** para que sea más rápido o consuma menos recursos, sin cambiar el resultado.

- *Ejemplo:* El optimizador se da cuenta de que la conversión de `60` a `60.0` se puede hacer de una vez durante la compilación, ahorrando una instrucción:

```
temp1 = id3 * 60.0 (Ahorramos la instrucción de conversión)
id1 = id2 + temp1
```

1.4.7 Generador de Código

Traduce el **código intermedio optimizado al lenguaje destino (código máquina o ensamblador)**. Aquí se asignan los **registros de memoria** reales de la CPU.

- *Ejemplo:*

```
LDF R2, id3      ; Cargar id3 en Registro 2
MULF R2, R2, #60.0 ; Multiplicar R2 por 60.0
LDF R1, id2      ; Cargar id2 en Registro 1
ADDF R1, R1, R2   ; Sumar R1 y R2
STF id1, R1      ; Guardar resultado en id1
```

1.4.8 La Tabla de Símbolos

Registra los **nombres** de los **elementos del programa fuente** (variables y procedimientos), junto con sus **atributos**, que contienen información de utilidad sobre cada nombre.

- Atributos de **variables**: dirección de almacenamiento, tipo, dimensión, alcance, precisión, si se ha declarado, si se ha inicializado, etc.
- Atributos de **procedimientos**: número y tipos de sus argumentos, modo de paso de cada argumento (valor o referencia), tipo devuelto, recursividad, etc.

1.5 Construcción de Compiladores

Para definir correctamente la construcción de un compilador, es imprescindible identificar tres lenguajes distintos:

- **El lenguaje fuente**: El lenguaje que el compilador traduce (entrada).
- **El lenguaje objeto**: El lenguaje al que se traduce y la plataforma donde se ejecutará (salida).
- **El lenguaje de implementación**: El lenguaje en el que está escrito el propio programa compilador.

Ejemplo: Si tenemos un ejecutable en un PC que traduce Pascal a código máquina:

- Fuente: Pascal.
- Objeto: Código máquina del PC.
- Implementación: Código máquina del PC (porque ya es un ejecutable).

1.5.1 Diagramas de Tombstone

Son una herramienta visual que facilita el **diseño y la comprensión de cómo interactúan compiladores, intérpretes y máquinas**. Hay cuatro tipos de piezas.

- **Compiladores (Forma de T)**: Representan la traducción de un lenguaje fuente a un lenguaje objeto o destino. En este diagrama, “Objeto” no se refiere necesariamente a un archivo `.o`; se refiere al **resultado de la traducción**, que puede ser ensamblador, código máquina, bytecode, código intermedio u otro lenguaje destino. En el caso concreto de C, un archivo `.o` suele contener código máquina relocalizable que todavía debe pasar por el enlazador.
- **Programas**: Representan un programa P escrito en un lenguaje L .
- **Máquinas**: Representan el hardware o sistema operativo base. Cuando ponen M es **máquina o plataforma**.
- **Intérpretes**: Representa el intérprete del lenguaje L escrito en M .

1.5.2 Reglas de Unión de Diagramas

La regla de oro para conectar estas piezas es: **Dos diagramas se pueden unir si en la unión los lenguajes son iguales.**

1.5.3 Estrategias Avanzadas de Construcción

Compilador- Enlazador

Divide la traducción en 2 fases:

- Traducir el lenguaje **fuentes a código intermedio**
- Traducir el **código intermedio** a **lenguaje máquina**

Se usa para desarrollar **múltiples compiladores** para **múltiples plataformas**.

Compilador Cruzado

Permite desarrollar compiladores para **maquinas diferentes a la de desarrollo**.

Bootstrapping

Se basa en **utilizar múltiples etapas de compilación** para mejorar la **expresividad** y **rendimiento** de un compilador. Una forma de bootstrapping consiste en **construir el compilador** de un lenguaje a partir de **una versión reducida del propio lenguaje**, de modo que se obtienen sucesivas versiones que **amplían y mejoran** un lenguaje de programación determinado.

Un **autocompilador** es aquel que es capaz de compilar su propio código fuente. Las **mayores ventajas** del bootstrapping se obtienen cuando un compilador se escribe en el **mismo lenguaje que compila**.

Compilador-Intérprete

Es la **unión del intérprete y la plataforma de ejecución** recibe el nombre de **maquina virtual**.

$$\text{Intérprete } (L \text{ en } M) + \text{Máquina } (M) \equiv \text{Máquina Virtual para } L$$

1.6 Aplicaciones

- Edición de textos con formato. Por ejemplo, LaTeX.
- Reconocimiento de patrones: tanto de texto, como reconocimiento del habla o visión por computadora.
- Desarrollo de editores de lenguajes estructurados. Por ejemplo, Xemacs. o Cálculo simbólico. Por ejemplo, MAPLE, SCILAB,...
- Diseño de circuitos integrados, mediante lenguajes como Verilog y VHDL.

- Traducción binaria. Para portar software entre plataformas.
- Simulación de arquitecturas hardware, para distintos conjuntos de datos, antes de su fabricación.

2. Análisis Léxico

El **Analizador Léxico (Scanner)** es el primer paso de la fase de análisis. Su trabajo es leer el archivo de texto (caracteres sueltos) y agruparlos en unidades con significado llamadas **tokens**.

NOTA

Token y **Componente léxico** son sinónimos y se usan indiferentemente por el profesor (creo).

Para mí un **componente léxico es una categoría**, por ejemplo un identificador o un número entero y un **token** es la representación que le entrega el analizador **léxico al sintáctico**

(`<identificador, "edad">`).

2.1 Estructura

El analizador sintáctico recibe los tokens y crea una estructura jerárquica que describe la **estructura gramatical** del código y léxico funcionan según el patrón productor-consumidor, siguiendo el siguiente esquema:

- **Consumidor (Sintáctico):** le pide al léxico el siguiente token.
- **Productor (Léxico):** lee los caracteres necesarios, forma el token y se lo entrega.

Tareas del analizador léxico:

- Reconoce los **componentes léxicos** de lenguaje.
- **Elimina** aquellos caracteres de código fuente **sin significado gramatical** (espacios en blanco, tabulaciones, comentarios, saltos de línea ...)
- Reconoce los **identificadores** de variables, tipo, constantes, métodos, etc. y los guarda en la **tabla de símbolos**.
- Avisa de **errores léxicos** detectados y los relaciona con un mensaje de error con el lugar en el que aparecen en el programa fuente.

Ventajas:

- Este modelo proporciona **simplicidad** porque el analizador sintáctico no tiene que preocuparse por espacios en blanco o comentarios.
- **Eficiencia** porque el análisis léxico consume mucho tiempo de CPU, al separarlo podemos usar técnicas de lectura rápida.
- **Portabilidad**, si cambias de caracteres (ej. de ASCII a Unicode), solo cambias el analizar léxico

Etapas del diseño de un analizador léxico:

- **Especificación** de los **términos del análisis léxico** mediante el uso de **expresiones regulares**.
- Diseño de un **autómata finito no determinista** que permita reconocer las expresiones regulares propuestas
- Realización de **autómata finito deterministas mínimo** que permita realizar un reconocimiento eficiente.
- Diseño y realización de un **sistema de entrada**.
- Diseño y realización de una **tabla de símbolos**.
- Diseño y realización de una estrategia de **manejo de errores**.

2.2 Especificación del Analizador

2.2.1 Términos del Análisis Léxico

- **Componente léxico (token): símbolo terminal** de la gramática que define el lenguaje fuente. Pueden ser signos de puntuación, operadores, palabras reservadas, identificadores ...
- **Patrón: expresión regular** que define el **conjunto de cadenas** correspondientes a un **componente léxico**. `[0-9]+` identifica cada una cadena de una o más cifras, correspondiente al componente léxico NÚMERO_ENTERO.
- **Lexema: cadena de caracteres** presente en el **código fuente** y que coincide con el **patrón de un componente léxico**. Por ejemplo, `453` coincide con el patrón de NÚMERO_ENTERO.
- **Atributos:** acompañan a cada **componente léxico** encontrado y permiten su **identificación y análisis posterior**. En la práctica un único atributo apunta a una entrada de la tabla de símbolos

Si por ejemplo tenemos la sentencia FORTRAN `E = C ** 2` se traduce como:

- <IDENTIFICADOR, apuntador en la tabla de símbolos a la entrada de E>
- <OP_ASIGNACIÓN>
- <IDENTIFICADOR, apuntador en la tabla de símbolos a la entrada de C>
- <OP_EXPONENTE>
- <NÚMERO_ENTERO, valor entero 2>

2.2.2 Expresiones Regulares

Una expresión regular r permite representar **patrones de caracteres**. El **conjunto de cadenas** representado por r recibe el nombre de **lenguaje generado** por r , y se escribe $L(r)$. Para un alfabeto Σ diremos que:

- $\emptyset$ es una expresión regular y $L(\emptyset) = \{\}$
- El símbolo ϵ (palabra vacía) es una expresión regular y $L(\epsilon) = \{\epsilon\}$
- Cualquier símbolo $a \in \Sigma$ es una expresión regular y $L(a) = \{a\}$

A partir de estas expresiones regulares básicas pueden construirse expresiones regulares más complejas aplicando las siguientes operaciones (vistas todas en talf):

- Concatenación
- Unión
- Cierre o clausura
- Cero o un caso
- Uno o más casos
- Clases de caracteres

Ejemplos:

- La expresión regular $a|b$ designa el conjunto $\{a, b\}$.
- La expresión regular $(a|b)(a|b)$ designa $\{aa, ab, ba, bb\}$. Otra expresión regular para el mismo conjunto es $aa|ab|ba|bb$. Se dice que ambas expresiones regulares son equivalentes.
- La expresión regular a^* designa $\{\varepsilon, a, aa, aaa, \dots\}$.
- La expresión regular $(a|b)^*$ designa el conjunto de todas las cadenas de a y b . Otra expresión regular equivalente es $(a * b^*)^*$.
- La expresión regular $a|a * b$ designa el conjunto que contiene la cadena a y todas aquellas con cero o más a seguidas de una b .

Por conveniencia, daremos nombres a las expresiones regulares, utilizando dichos nombres **como si fuesen símbolos**:

$$d \rightarrow r$$

Ejemplo: Definimos los números sin signo en Pascal como:

$$digito \rightarrow 0|1|\dots|9$$

$$digitos \rightarrow digito\ digito^*$$

$$fracción \rightarrow .digito|\epsilon$$

$$exponente \rightarrow (E(+|-|\epsilon)digitos)|epsilon$$

$$numero \rightarrow digitos\ fraccion\ exponente$$

2.3 Autómatas Finitos

Lo mismo que vimos en **Teoría de Autómatas y Lenguajes formales**:

2.3.1 AFN

Para construir un AFN de un modo sistemático empleamos la **Construcción de Thompson**, que utiliza tres reglas básicas:

- Para el símbolo ϵ , se construye el AFN siguiente:

- Para el símbolo $a \in \Sigma$, se construye el AFN siguiente:
- Supongamos que $N(s)$ y $N(t)$ son AFN para las expresiones regulares s y t :
 - Para la expresión regular $s|t$ se construye el AFN $N(s|t)$
- Para la expresión regular st se construye el AFN $N(st)$.
- Para la expresión regular s^* se construye el AFN $N(s^*)$ siguiente:

Se puede demostrar que la aplicación sistemática de las reglas anteriores da lugar a un **AFN** $N(r)$ que **reconoce la expresión regular** inicial r . Este AFN tiene las siguientes **propiedades**:

- $N(r)$ tiene un **estado inicial** y uno **final**. Esa propiedad la satisfacen asimismo todos los AFN que lo componen
- $N(r)$ tiene a lo sumo el **doble de estados** que **de símbolos y operadores** en r (cada vez que se aplica una regla se crean a lo sumo dos nuevos estados)
- Cada estado de $N(r)$ tiene una **transición saliente** con un símbolo $A \in \Sigma$, o a lo sumo dos transiciones salientes con símbolos ϵ .

2.3.2 Autómata Finito Determinista

Necesitamos convertir el **AFN** a un **AFD** para ello empleamos el método de **construcción de subconjuntos** para obtener el AFD equivalente a un AFN dado. Este AFD será más sencillo de programar. Este algoritmo lo vimos en **TALF**.

2.3.3 AFD Mínimo Equivalente

Un analizador léxico será más eficiente cuanto menor sea el número de estados del AFD correspondiente. Para cualquier AFD existe un AFD mínimo equivalente.

2.3.4 Construcción del Analizador

El analizador léxico se construirá a partir de la agregación de AFD organizados según un orden conveniente. Siempre se debe reconocer la cadena más larga. El último carácter leído será devuelto al flujo para ser leído en la siguiente iteración.

Podemos integrar los tres autómatas anteriores en el siguiente autómata:

2.3.5 Herramienta LEX

LEX es una herramienta diseñada para **generar analizadores léxicos automáticamente** a partir de una especificación basada en el uso de expresiones regulares.

El fichero de especificación permite **describir los componentes léxicos** del lenguaje fuente y las acciones a realizar. Su compilación da lugar a una función `yyllex()` que invoca el analizador sintáctico para leer componentes léxicos.

2.4 Sistema de Entrada

El **sistema de entrada** es un conjunto de rutinas que interactúan con el sistema operativo para la **lectura** de datos del **programa fuente**. El sistema de entrada y el analizador léxico funcionan según el patrón productor-consumidor.

La separación del sistema de entrada supone una **mejora** en:

1. **Eficiencia:** un analizador léxico en C tiene que acceder a un carácter de un fichero, que pasará del disco a la memoria, luego a una estructura file y por último a un string del analizador.
2. **Portabilidad:** como el sistema de entrada es el único componente del compilador que se comunica con el SO, para cambiar de plataforma solo hay que cambiar el sistema de entrada.

El analizador léxico debe detectar el componente léxico con el **lexema más largo posible**, por lo que hay que **leer caracteres anticipadamente**. Para resolver esto se usa una **memoria intermedia** que pueda **almacenar** un **bloque de caracteres** de disco y **apuntar** el fragmento **ya analizado**. Al **devolver** caracteres al flujo de entrada se pueda **mover un puntero** tantas posiciones como caracteres a devolver.

Cualquier sistema de entrada debe:

- Ser lo más **rápido** posible
- Permitir un **acceso eficiente a disco**
- Hacer **uso eficiente** de la **memoria**
- Soportar **lexemas** de **longitud considerable**
- Tener disponibles el **lexema actual y el anterior**.

2.4.1 Método del Par de Memorias Intermedias

El **método del par de memorias intermedias** divide la memoria intermedia en **dos mitades** de n bytes cada una (n debe ser múltiplo de la longitud de la **unidad de asignación**).

0. Al principio, los punteros `inicio` y `delantero` apuntan al primer carácter de un lexema.
1. A medida que el analizador pide caracteres, `delantero` se mueve hacia delante:
 1. Se comprueba si se ha alcanzado el `EOF`.
 2. Se comprueba si se ha alcanzado el final de un bloque. Funciona como una lista circular, si se sobrepasa el bloque *A* se lee *B*, si sobrepasa *B* se lee *A*.
2. Tras reconocer un componente léxico, `inicio` avanza hasta la posición de `delantero` y se inicia el análisis del siguiente lexema.

Limitaciones:

1. **Tamaño del lexema limitado por n :** Aunque en condiciones ideales un lexema podría ocupar hasta $2n$, el sistema solo garantiza el reconocimiento de lexemas de longitud máxima n . Si un lexema supera este tamaño, al avanzar el puntero `delantero` hacia un nuevo bloque, se sobrescribirá la memoria donde se encontraba el puntero `inicio`, perdiendo el comienzo del lexema antes de ser procesado.
2. **Ineficiencia en el avance:** Por cada carácter que avanza el puntero `delantero`, el sistema debe realizar **tres comprobaciones lógicas**: si es fin de fichero, si es fin del bloque A o si es fin del bloque B.

2.4.2 Método del Centinela

En el método del **centinela** se añade un byte más a cada bloque, en el que se guardará un carácter centinela (EOF). De este modo, se hace una sola comprobación lógica cada vez que avanza `delantero`. Si la comprobación es positiva, se analiza cuál de los tres casos es.

2.5 Tabla de Símbolos

La **tabla de símbolos** es la estructura de datos utilizada por el compilador para **gestionar los identificadores** que aparecen en el programa fuente: constantes, variables, tipos, funciones,...

1. Cuando el compilador **encuentra un identificador**, **guarda** en la tabla información que lo caracteriza.
2. Cuando se **referencia** este identificador, el compilador **consulta** la tabla de símbolos y obtiene la información que necesita.
3. Una vez **fuera del ámbito** del identificador, se **elimina** de la tabla.

Durante la compilación:

1. El **analizador léxico**, cuando **encuentra** un identificador, **comprueba** que está en la tabla de símbolos. Si **no lo está**, **crea** una nueva entrada para el mismo.
2. El **analizador sintáctico** añade información a los campos de los **atributos**. Pero también puede crear **nuevas entradas** si se definen nuevos **tipos** de datos como palabras reservadas.
3. El **análisis semántico** debe acceder a la tabla para **consultar los tipos de datos** de los símbolos.

4. El **generador de código** puede:

1. leer el **tipo de dato** de una variable para la reserva de espacio;
2. **guardar la dirección** de memoria en la que se almacenará una variable.

2.5.1 Palabras Reservadas

Algunos lenguajes **reservan** algunas palabras que **no pueden utilizarse como identificadores**:

`printf, for, if, while`, etc. **Pueden distinguirse**:

1. Definiéndolas mediante **expresiones regulares**, y asociándoles un componente léxico particular.
2. Mediante una **tabla de palabras clave**. Cada vez que se encuentra un lexema correspondiente a un identificador se busca en esta tabla.
3. Insertándolas al principio de la **tabla de símbolos**. Se gestionan en la misma tabla palabras clave e identificadores.

La tabla de símbolos se inicializa en las **primeras posiciones** con las palabras reservadas del lenguaje, ordenadas por **orden alfabético**. Cuando se encuentra un nombre en el código fuente se **consulta** la tabla de símbolos:

- Si **se encuentra entre las palabras reservadas se devuelve su componente léxico al analizador sintáctico**.
- Si se encuentra **después de las palabras reservadas es un identificador previamente encontrado**.
- Si no se encuentra **en** la tabla de símbolos, se añade como un **nuevo** identificador.

De este modo se **reduce el tamaño del AFD y se aumenta la eficiencia del análisis léxico**.

2.5.2 Estructura de la Tabla de Símbolos.

La **tabla de símbolos** se estructura en un conjunto de **registros**, cuya longitud suele ser fija, conteniendo el **lexema** encontrado y un conjunto de atributos para su componente léxico. Funciona como una **base de datos** en la que el campo clave es el **lexema** del símbolo. Hay dos formas de almacenamiento:

- **Estructura Interna:** Si reservas 32 caracteres para cada nombre y el programador usa variables cortas como `i` o `x`, **desperdicias mucho espacio**. Si el programador usa un nombre de 33 caracteres, **lo cortas** y causas errores.
- **Estructura Externa:** Esta estructura **no exige una longitud fija para los identificadores**, por lo que se aprovecha mejor el espacio de almacenamiento.

Las **operaciones** que se ejecutan sobre la tabla son:

- **Buscar** un lexema y el contenido de sus atributos.
- **Insertar** un nuevo registro previa comprobación de su no existencia.

- **Modificar** la información contenida en un registro. En general, se realiza una operación de adición de información.

Además, en lenguajes con estructura de bloque:

- **Nuevo bloque:** comienzo de un nuevo bloque.
- **Fin de bloque:** final de un bloque.

2.5.3 Organización de la Tabla

Las tablas de símbolos se organizan de dos maneras:

- **Tablas no ordenadas.** Generadas con vectores o listas. Poco eficientes pero fáciles de programar.
- **Tablas ordenadas.** Permiten definir el tipo diccionario. Utilizan estructuras de tipo vector o lista ordenada, árboles binarios, árboles equilibrados (AVL), tablas de dispersión (hash), etc.

Las **reglas de ámbito**, propias de cada lenguaje, determinan la definición de cada lexema en cada momento de la compilación.

Tablas de Símbolos no Ordenadas

Las **tablas de símbolos no ordenadas** se usa un vector o lista y una pila auxiliar de **punteros de índice de bloque** para marcar el comienzo de los símbolos que corresponden a un bloque.

Al **terminar un bloque** se eliminan todos los símbolos desde el siguiente al apuntado hasta el final de la tabla de símbolos.

Admite las siguientes **operaciones**:

- **Inserción:** al encontrar la declaración de un símbolo se debe verificar que no se encuentra en el último bloque. Si no lo está, se inserta en la última posición de la tabla, si lo está se devuelve un error (ya que el símbolo ya está definido en ese ámbito)
- **Búsqueda:** se hace desde el final de la tabla hasta el principio. Si se encuentra, se corresponde a la declaración realizada en el bloque más próximo.
- **Nuevo bloque:** se añade en la pila un puntero al último símbolo de la tabla
- **Fin de bloque:** se eliminan todos los símbolos de dicho bloque desde el siguiente al de inicio y después se elimina el índice de la pila.

Tabla de Símbolos con Estructura de Árbol

Se usa un **árbol binario ordenado** y se añade un **campo** a cada registro con el **nivel** al que pertenece el símbolo. Para evitar duplicidades se usan **listas encadenadas**.

- **Inserción:** se busca la posición que le corresponde. Si ya hay un registro con el mismo lexema, se comprueba que el campo del nivel no tiene el mismo valor que el bloque activo. Si fuese así se

indica error, en caso contrario se añade el nuevo nodo a la lista del primero. Si no hay registro se inserta en el árbol.

- **Búsqueda:** la propia de un árbol binario
- **Nuevo bloque:** se incrementa en 1 el campo de nivel
- **Fin de bloque:** se eliminan los símbolos de dicho bloque borrando sus registros del árbol y después decrementando en 1 la variable con el número de nivel.

Tabla de Símbolos con Estructura de Bosque

Se usa un **árbol binario** para **cada bloque** y una **pila de índices de nivel** que apuntan a la raíz del árbol del nivel.

- **Inserción:** se realiza una inserción normal en el árbol del bloque activo
- **Búsqueda:** se busca el lexema en el árbol del bloque activo. Si no se encuentra, se busca en el árbol del bloque anterior, y así sucesivamente.
- **Nuevo bloque:** se crea un nuevo elemento en la pila de índices y una nueva estructura de tipo árbol
- **Fin de bloque:** se destruye el árbol asociado y se elimina el último puntero a la pila de índices.

Tabla Hash

Aplica una **función matemática** al lexema para determinar la **posición** de la tabla que le corresponde. Existen dos opciones para **manejar las colisiones** que se producen cuando dos lexemas ocuparían la misma posición:

- Tablas de dispersión **cerradas:** cuando se produce una colisión se usa alguna técnica que permita acceder a una posición vacía (sondeo lineal, multiplicativo, cuadrático, etc.)
- Tablas de dispersión **abiertas:** se usa una lista encadenada de desbordamiento para resolver colisiones.
- **Inserción:** como en cualquier tabla hash (se suelen usar listas de desbordamiento con los símbolos más recientes en las primeras posiciones).
- **Búsqueda:** como en cualquier tabla
- **Nuevo bloque:** sólo hay que almacenar el cambio de bloque activo
- **Fin de bloque:** se borran todos los registros correspondientes a este bloque recorriendo prácticamente toda la tabla y después se cambia el bloque activo.

Análisis de Complejidad

2.6 Tratamiento de Errores

Cuando en un momento del proceso de compilación se detecta un error:

- Se debe mostrar un **mensaje claro y exacto**, que permita al programador encontrar y corregir fácilmente dicho error.
- Debe **recuperarse del error** e ir a un estado que permita continuar analizando el programa en búsqueda de otros errores. Se debe evitar una cascada de errores, o pasar por alto otros.
- **No debe retrasar** excesivamente el procesamiento de programas correctos.

El proceso de recuperación de un error puede suponer la adopción de diferentes medidas:

- **Ignorar** los caracteres inválidos hasta formar un componente léxico correcto.
- **Eliminar** caracteres que dan lugar a error.
- **Intentar corregir** el error:
 - **Insertar** los caracteres que pueden faltar.
 - **Reemplazar** un carácter presuntamente incorrecto por uno correcto.
 - **Intercambiar** caracteres adyacentes.
- **Cuidado**, intentar corregir un error puede ser peligroso.

3. Análisis Sintáctico

3.1 Nociones Generales

3.1.1 Gramáticas Independientes del Contexto

Una gramática independiente del contexto se define como una 4-tupla:

$$G = (\Sigma, V, S, P)$$

donde:

- Σ es el conjunto de **símbolos terminales**.
- V es el conjunto de **símbolos no terminales** o variables sintácticas.
- $S \in V$ es el **símbolo inicial**.
- P es el conjunto de **producciones**, de la forma:

$$A \rightarrow \alpha$$

con $A \in V$ y $\alpha \in (V \cup \Sigma)^*$.

Observación: en algunos apuntes también se escribe $G = (NT, T, P, S)$; es la misma idea, cambiando la notación.

La máquina teórica asociada al reconocimiento de los lenguajes independientes del contexto es el **autómata a pila**.

En este tema se usa además el símbolo $\$$ para indicar **fin de cadena**.

3.1.2 Tipos de Analizadores Sintácticos

Un **analizador sintáctico** analiza el **código fuente** como una **secuencia** de **componentes léxicos** (símbolos terminales) y construye una representación interna en forma de **árbol sintáctico**. Podemos hablar de dos tipos de analizadores:

- **Descendentes:** parten de la raíz del árbol, aplican las reglas de la gramática **sustituyendo la parte izquierda** de una regla por su **parte derecha**.
- **Ascendentes:** parten de las hojas, aplican las reglas de la gramática **reduciendo la parte derecha** de una regla por su **parte izquierda**.

3.1.3 Forma de Backus-Naur Extendida

La **BNF extendida** es la **notación** más común para **fomalizar gramáticas**.

Notación Básica:

- `::=` usada en **definiciones**.
- `|` usada para representar **disyunción**
- `<>` usada para representar **símbolos NO TERMINALES**.

Notación Extendida:

- `[]` usados para representar **símbolos opcionales** (pueden aparecer una vez o ninguna).
- `{ }` usados para representar **repetición** cero, una o más veces.
- `"` usados para **distinguir metasímbolos de terminales**

Notación Visual:

- Los símbolos **terminales** se marcan en **negrita**
- Se **eliminan** los símbolos `<>` de los **no terminales**
- Se utiliza el símbolo `->` en vez de `::=`

3.2 Diseño de una Gramática

Al desarrollar un **nuevo lenguaje** de programación, se debe diseñar una **nueva gramática** que **describirá sus características**. Algunos aspectos que deben ser discutidos en el diseño de una nueva gramática son:

- **Recursividad:** lo que permite **reducir** el número de **reglas sintácticas**
- **Ambigüedad:** ha de **evitarse** pues proporciona una **gramática** a menudo **más intuitiva**, pero permite generar **distintos** códigos **objeto** para el **mismo** código **fuentes**.
- **Asociatividad:** se deben proporcionar **reglas de asociatividad** para las **distintas expresiones** del lenguaje
- **Precedencia:** determina el **orden** en el que se realizarán las **distintas operaciones** del lenguaje.

3.2.1 Recursividad

Los lenguajes de programación pueden generar **infinitos programas**, así que hay que usar un **método de generación** sin realizar una **casuística interminable**. La **recursividad** permite generar un número **infinito de programas** mediante una **gramática finita**.

Una gramática es **recursiva** si, partiendo de un **NT** (A) y aplicando las reglas una o varias veces ($\Rightarrow^+$), terminamos con una cadena que contiene ese **NT** rodeado por otras cosas ($\alpha A \beta$) $\rightarrow A \Rightarrow^+ \alpha A \beta$

3.2.2 Ambigüedad

Una gramática es **ambigua** si genera al menos una cadena con **dos o más árboles de derivación distintos**. Se debe **evitar** la ambigüedad, ya que pudiendo utilizar distintos árboles sintácticos para genera una misma sentencia, el **código generado** podrá ser **distinto** en **distintas ejecuciones** del compilador.

Factorización por la izquierda

Cuando en el diseño de una gramática aparecen **dos producciones** de un **mismo No Terminal** que **empiezan igual**, no se sabrá cuál de las reglas elegir.

La **factorización por la izquierda** solucione esto **reescribiendo** dichas reglas para **retrasar la decisión** hasta haber visto suficiente. En general, si nuestra gramática tiene una producción de la forma $A \rightarrow \alpha\beta_1|\alpha\beta_2|\dots|\alpha\beta_n|Y$

1. Realizar la sustitución siguiente:
 - $A \rightarrow \alpha A' | Y$
 - $A' \rightarrow \beta_1|\beta_2|\dots|\beta_n$
2. Comprobar si $\beta_1|\beta_2|\dots|\beta_n$ se pueden factorizar

A = un no terminal
 α = prefijo común
 $\beta_1\dots\beta_n$ = continuaciones distintas después del prefijo común
 Y = otra alternativa que no empieza por α

A = <sentencia>
 α = if <expresion> then <sentencia>
 β_1 = ϵ
 β_2 = else <sentencia>

3.2.3 Asociatividad

Cuando diseñamos un lenguaje debemos decidir el tipo de **asociatividad** de las **operaciones** entre **más de dos operandos**.

- Asociatividad por la **derecha**: las operaciones se hacen de derecha a izquierda. $a = b = c \equiv a = (b = c)$
- Asociatividad por la **izquierda**: las operaciones se hacen de izquierda a derecha. $a + b + c \equiv (a + b) + c$

La asociatividad se puede incorporar a una gramática mediante la **recursividad**, distinguiendo entre:

- Operador asociativo por la **derecha**: una regla sintáctica en la que aparece el operador se hace recursiva por la derecha. `<asignacion> ::= id = <asignacion> | id`
- Operador asociativo por la **izquierda**: la regla sintáctica en la que aparece el operador se hace recursiva por la izquierda. `<expresion> ::= <expresion> + id | id`

3.2.4 Precedencia

La **precedencia** permite especificar un **orden** relativo de **evaluación** de unos operadores respecto a otros. Se puede incorporar usando:

- Una **variable sintáctica** para cada **nivel** de precedencia
- Una **producción** para cada **operador**.

Un operador tendrá **menos precedencia** cuanto más **cerca** esté su regla de derivación de la **regla inicial** de la gramática. A mayor precedencia mayor prioridad de agrupación/evaluación.

3.3 Análisis Sintáctico Descendente

El **análisis sintáctico** consiste en, una vez definida una gramática, comprobar que cada programa escrito en código fuente obedece sus reglas.

3.3.1 ASD Recursivo

El caso más simple de ASD (Análisis Sintáctico Descendente) es totalmente **recursivo**:

- **Avanzamos**: cuando sustituimos un **no terminal** por alguna de sus producciones
- **Retrocedemos**: cuando se llega a un punto muerto y es necesario probar otra regla de sustitución.

Para gestionar este proceso usamos una **pila** en la que almacenamos los **símbolos** de cada sustitución, tratando de **emparejar** el símbolo que hay en la **cima** de la pila con la **entrada actual**.

Si al final del proceso la **pila** y la **entrada** están **vacías**, entonces la sentencia pertenece al lenguaje.

Una posible implementación del ASD se puede realizar desarrollando una **función de análisis por cada no terminal**.

- Son muy **lentos** debido a los **retrocesos**
- No se sabe si la entrada pertenece o no al lenguaje hasta **analizar todas las posibilidades**. Si no pertenece o no al lenguaje hasta **analizar todas las posibilidades**. Si no pertenece, en la pila tenemos el símbolo S , por lo que no podemos decir **dónde está el error**.

- Si se genera código objeto durante el análisis, en cada **retroceso** hay que **eliminar parte del código generado**.
- Estos inconvenientes hacen que estos métodos de ASD **no sean recomendables**.

Recursividad por la izquierda

Las **reglas recursivas por la izquierda** pueden hacer entrar al ASD en un **bucle infinito**. Por tanto, hay que **eliminar** la recursividad por la izquierda.

Sea una gramática recursiva: $A \rightarrow A\alpha_1 | A\alpha_2 | \dots | A\alpha_n | \beta_1 | \beta_2 | \dots | \beta_n$. Una **gramática equivalente no recursiva por la izquierda** será:

- $A \rightarrow \beta_1 A' | \beta_2 A' | \dots | \beta_n A'$
- $A' \rightarrow \alpha_1 A' | \alpha_2 A' | \dots | \alpha_n A' | \epsilon$

$\alpha_1, \alpha_2, \dots$ = alternativas recursivas por la izquierda
 $\beta_1, \beta_2, \dots$ = alternativas no recursivas

Por ejemplo

$A \rightarrow A + T \mid A - T \mid T$

Dónde:

$A = A$
 $\alpha_1 = + T$
 $\alpha_2 = - T$
 $\beta_1 = T$

Sigue siendo recursiva y es equivalente a la gramática anterior, pero la A ya no se encuentra a la izquierda (evita bucles infinitos.)

Este método elimina la recursividad inmediata (es decir, en la misma derivación). Si la recursividad aparece en **derivaciones posteriores** hay que encontrar el **elemento conflictivo** y **sustituirlo** por su definición.

3.3.2 ASD Predictivo

Pretenden **evitar** los **retrocesos prediciendo** en cada momento cuál de las **reglas** se debe aplicar para continuar el análisis correctamente. Supongamos que la cadena de entrada es $c_1, \dots, c_n$, el no terminal a sustituir es A y que las sustituciones posibles son $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$

Si **cada regla** de sustitución **comienza** por un **componente** léxico **distinto** el análisis **nunca retrocederá**, salvo error. Debemos aplicar la **regla de sustitución** que **comienza** por c_i (componente actual).

Gramáticas LL(k)

Las gramáticas que se proponen son las $LL(k)$.

- L (left): la entrada se lee de **izquierda a derecha**.
- L (left): se sustituye el **no terminal** situado a la **izquierda**
- (k) : se leen k **componentes** léxicos por **anticipado**

Nos centramos en gramáticas $LL(1)$, que debe cumplir:

- **No** puede ser **recursiva por la izquierda**
- **Dos reglas** de sustitución de un **mismo no terminal** no pueden dar lugar al **mismo componente léxico inicial**.
- Estas condiciones son **necesarias no suficientes**

Si encontramos una gramática no $LL(1)$ la modificaremos:

- **Eliminando la recursividad por la izquierda**
- Sacando **factor común por la izquierda**.

Conjuntos de Predicción

Definimos dos **conjuntos de predicción**, que ayudarán en el proceso de análisis predictivo: el conjunto de **primeros** y el conjunto de **siguientes**.

Conjunto Primeros

Sea X un símbolo gramatical ($X \in (V \cup \Sigma)$), $PRIMEROS(X)$ es el conjunto de **terminales** (incluyendo ϵ) que aparecen al **inicio** de las **cadenas derivables de X** en ninguno o más pasos.

$$PRIMEROS(X) = \{v | X \Rightarrow^* v\beta, v \in \mathcal{E}, \beta \in \mathcal{E}^*\}$$

Para calcular $PRIMEROS(X)$:

1. Si X es un símbolo terminal a , $PRIMEROS(X) = a$
2. Si $(X \rightarrow \epsilon) \in P$ añadimos ϵ a $PRIMEROS(X)$
3. Si X es un **no terminal** y $X \rightarrow Y_1, Y_2, \dots, Y_k$ es una regla de sustitución, añadimos $PRIMEROS(Y_j)$ a $PRIMEROS(X)$, si $PRIMEROS(Y_j)$ es un terminal, y para todo $i = 1, 2, \dots, j - 1$ $Y_i \Rightarrow^* \epsilon$
4. Si X es un **no terminal**, y $X \rightarrow Y_1 Y_2 \dots Y_k$ es una regla de sustitución, y $Y_j \Rightarrow^* \epsilon \forall j \in 1, 2, \dots, k$ añadimos ϵ a $PRIMEROS(X)$

Para calcular $PRIMEROS(X_1, X_2, \dots, X_n)$:

1. Añadimos todos los símbolos, distintos de ϵ de $PRIMEROS(X_1)$
2. Si ϵ está en $PRIMEROS(X_1)$, añadir también los símbolos distintos de ϵ de $PRIMEROS(X_2)$
3. Si ϵ está en $PRIMEROS(X_2)$, añadir también los símbolos distintos de ϵ de $PRIMEROS(X_3)$ y así sucesivamente.
4. Por último, añadir ϵ a $PRIMEROS(X_1, X_2, \dots, X_n)$ si $PRIMEROS(X_i)$ contiene a ϵ , $\forall i \in 1, 2, \dots, n$

Traducción: Los **conjuntos PRIMEROS**, o **FIRST**, sirven para responder esta pregunta: Si empiezo a derivar desde un símbolo o una secuencia de símbolos, ¿qué terminales pueden aparecer como primer token?

Por ejemplo, si tienes:

$$A \rightarrow a B \mid b C$$

entonces:

$$PRIMEROS(A) = \{ a, b \}$$

Si X es un **terminal**, entonces su conjunto de primeros es él mismo.

$$\begin{aligned} PRIMEROS(id) &= \{ id \} \\ PRIMEROS(+) &= \{ + \} \\ PRIMEROS(*) &= \{ * \} \\ PRIMEROS(() &= \{ (\} \end{aligned}$$

Si X es un **no terminal**, si tienes:

$$A \rightarrow a B \mid b C \mid \epsilon$$

entonces miras cómo puede empezar cada alternativa:

$$\begin{aligned} A \rightarrow a B & \text{ empieza por } a \\ A \rightarrow b C & \text{ empieza por } b \\ A \rightarrow \epsilon & \text{ puede desaparecer} \end{aligned}$$

Por tanto:

$$PRIMEROS(A) = \{ a, b, \epsilon \}$$

Regla mental para calcular $PRIMEROS(X_1, X_2, \dots, X_n)$:

Empieza por X_1 .
Añade sus $PRIMEROS$ excepto ϵ .
Si X_1 puede ser ϵ , mira X_2 .
Añade sus $PRIMEROS$ excepto ϵ .
Si X_2 puede ser ϵ , mira X_3 .
Y así sucesivamente.
Solo añades ϵ si todos pueden desaparecer.

Ejemplo:

$PRIMEROS(A) = \{ a, \epsilon \}$
 $PRIMEROS(B) = \{ b, \epsilon \}$
 $PRIMEROS(C) = \{ c \}$

Entonces:

$PRIMEROS(A B C) = \{ a, b, c \}$

Si por ejemplo quisiese:

$PRIMEROS(A B)$

Entonces:

$PRIMEROS(A B) = \{ a, b, \epsilon \}$

Conjunto Siguietes

Sea A un símbolo **no terminal**, $SIGUIENTES(A)$ es el conjunto de **terminales** que pueden aparecer **inmediatamente a la derecha de A en alguno sustitución**.

$$SIGUIENTES(A) = \{ v \mid S \Rightarrow^* \alpha A v \beta, v \in \mathcal{E}, \alpha, \beta \in \mathcal{E}^* \}$$

Si A aparece al **final de una cadena** de símbolos, entonces diremos que $\$ \in SIGUIENTES(A)$

Para calcular $SIGUIENTES(A)$:

1. Si A es S , añadir $\$$ a $SIGUIENTES(S)$.
2. Si para cada regla $B \rightarrow \alpha A \beta$, añadir $PRIMEROS(\beta) - \epsilon$ a $SIGUIENTES(A)$
3. Para cada regla $B \rightarrow \alpha A \beta$, donde $PRIMEROS(\beta)$ contiene a ϵ ($\beta \Rightarrow^* \epsilon$), añadir todos los símbolos de $SIGUIENTES(B)$ a $SIGUIENTES(A)$

4. Para cada regla $B \rightarrow \alpha A$ añadir $SIGUIENTES(B)$ a $SIGUIENTES(A)$

Traducción: El conjunto **SIGUIENTES(A)**, también llamado **FOLLOW(A)**, contiene los terminales que pueden aparecer **inmediatamente después** del no terminal **A** en alguna derivación.

1. Si **S** es el símbolo inicial:

$\$ \in SIGUIENTES(S)$

2. Si aparece una producción:

$B \rightarrow \alpha A \beta$

entonces se añade:

$PRIMEROS(\beta) - \{ \epsilon \}$ a $SIGUIENTES(A)$

3. Si en:

$B \rightarrow \alpha A \beta$

la parte **β** puede derivar en vacío, es decir:

$\beta \Rightarrow^* \epsilon$

entonces se añade:

$SIGUIENTES(B)$ a $SIGUIENTES(A)$

4. Si **A** aparece al final de una producción:

$B \rightarrow \alpha A$

entonces se añade:

$SIGUIENTES(B)$ a $SIGUIENTES(A)$

Tabla de Análisis Sintáctico

La **tabla de análisis sintáctico LL(1)** sirve para que un parser predictivo sepa **qué regla aplicar** sin probar ni retroceder. La idea es:

Estoy intentando expandir un no terminal A
y el siguiente token de la entrada es x .

¿Qué producción de A debo usar?

Mediante los conjuntos de predicción podemos construir una tabla que **indique qué regla se debe aplicar** dado el **no terminal** a expandir y el **componente léxico actual**. En esta tabla de **filas** son símbolos **no terminales** de la gramática, y sus **columnas** son los **componentes léxicos** junto al símbolo de **fin de cadena** $\$$. En las **celdas** de la tabla aparecen **reglas** de la gramática, de manera que las **celdas vacías** se corresponden con un **error en el análisis**.

Una gramática es $LL(1)$ si en la **tabla de análisis sintáctico** aparece como máximo una regla en cada celda.

Las celdas de la tabla $T[A, a]$ se rellenan siguiendo el siguiente procedimiento para cada regla de sustitución $A \rightarrow \alpha$:

1. Para cada terminal $a \in \text{PRIMEROS}(\alpha)$, añadir $A \rightarrow \alpha$ en $T[A, a]$
2. Si $\epsilon \in \text{PRIMEROS}(\alpha)$ para cada terminal $b \in \text{SIGUIENTES}(A)$, añadir $A \rightarrow \alpha$ en $T[A, b]$.

Traducción: Para cada producción:

$A \rightarrow \alpha$

se hace esto:

1. Calcula $\text{PRIMEROS}(\alpha)$.
2. Para cada terminal a en $\text{PRIMEROS}(\alpha)$, coloca $A \rightarrow \alpha$ en $T[A, a]$.
3. Si ϵ está en $\text{PRIMEROS}(\alpha)$, entonces coloca $A \rightarrow \alpha$ en $T[A, b]$ para cada b en $\text{SIGUIENTES}(A)$.

Procedimiento de Análisis

Podemos proporcionar un **procedimiento de análisis** basado en el uso de una **tabla de análisis sintáctico** para **seleccionar la siguiente regla** de sustitución y una **pila** para almacenar los **símbolos gramaticales**.

El algoritmo de análisis es el siguiente:

1. Inicializar la pila con el símbolo S y añadir $\$$ a continuación y al final de la cadena de entrada
2. Comparar el símbolo X de la cima de la pila y el siguiente símbolo a de la entrada.
 1. Si $X = a = \$$ aceptar la cadena y salir
 2. Si $X = a \neq \$$ extraer el elemento de la pila y avanzar una posición en la cadena de entrada
 3. Si X es un **terminal** $\neq a$ o X es un **no terminal** y $T(X, a)$ está vacía es un error.
 4. Si X es un **no terminal** y en $T(X, a)$ tenemos $X \rightarrow X_1 X_2 \dots X_n$ extraer X de la pila a insertar $X_1 X_2 \dots X_n$
 5. Volver al paso 2.

Resumen

Las **gramáticas** $LL(1)$ admiten un proceso de **análisis** de complejidad $O(n)$. No todas las gramáticas son $LL(1)$, pero en algunos casos se pueden llevar a cabo ciertas **transformaciones para convertirlas en $LL(1)$** :

- **Eliminar la recursividad por la izquierda**
- **Eliminar la ambigüedad**
- **Factorizar por la izquierda**

3.3.3 Gestión de errores en ASD

Pueden darse dos tipos de errores en el ASD:

- En la parte **superior de la pila** hay un **terminal** que **no** se corresponde con el **componente léxico actual**: el compilador deberá señalar el error, indicando qué es lo que esperaba encontrarse.
- En la parte **superior de la pila** hay un **no terminal**, y la **celda** de la **tabla** correspondiente al **componente léxico actual** está **vacía**: el compilador deberá señalar un error conforme se esperaba uno de los componentes léxicos del conjunto de celdas del **no terminal** que contienen algún dato.

Al **detectar** el error el compilador debe **continuar el análisis** para lo que puede aplicar la técnica de **conjuntos de sincronización**. Esta sincronización se basa en que, al detectar un error:

1. Se entra en un **estado de error**
2. Se **continúa analizando** la entrada, **descartando** los **componentes léxicos** que aparecen hasta **encontrar** uno que pertenezca al **conjunto de sincronización**
3. Se **valora quitar** el símbolo de **encima de la pila**, se **abandona el estados de error** y se **prosigue** con el análisis.

La **efectividad** de este método depende de la **elección del conjunto de símbolos de sincronización**. Normalmente se utilizan **técnicas empíricas**, basadas en el estudio de los errores **más frecuentes**.

- Para un **no terminal** A , se suelen incluir en su conjunto de sincronización los símbolos de $SIGUIENTES(A)$, ya que, si aparece uno de esos símbolos, es probable que el análisis pueda continuar dando por terminado A y quitándolo de la pila.
- En los lenguajes de programación suele haber niveles jerárquicos, como **expresión**, **sentencia**, **bloque**, etc. Por ello, se pueden añadir al conjunto de sincronización de un **no terminal de baja jerarquía** los símbolos que inician construcciones de **nivel superior**, para poder recuperar el análisis en un punto más estable.
- Para un **no terminal** A , también se pueden añadir los símbolos de $PRIMEROS(A)$ a su **conjunto de sincronización**, ya que indican posibles comienzos válidos de A y pueden servir para retomar el análisis.
- Si un no terminal puede generar ϵ , se puede tomar esta producción por **omisión**, dejando la detección y recuperación del error para símbolos más importantes o de mayor jerarquía.
- Si el error se produce porque no se puede **emparejar un terminal**, se puede **extraer dicho terminal de la pila** como si hubiese aparecido en la entrada, emitir un mensaje de error indicando que se esperaba ese símbolo y proseguir con el análisis.

3.4 Análisis Sintáctico Ascendente

El **análisis sintáctico ascendente** parte de una cadena de entrada formada por una secuencia de **componentes léxicos** y busca **corresponder las partes derechas** de las **reglas** de la gramática. Cuando encuentra una, la **sustituye** (decimos reduce) por su **parte izquierda**, y crea un **nuevo nodo** del árbol sintáctico. Continúa así hasta llegar a S o **error**.

Puede haber **distintas reglas** de sustitución con alguna **parte derecha común**, y reglas que sustituyan por la **cadena vacía**. El problema es **decidir** cuándo lo que parece una parte derecha de una regla se puede sustituir por su parte izquierda.

Los algoritmos de **análisis sintáctico ascendente** utilizan una **pila** en su análisis y realizan dos **operaciones** básicas:

- **Desplazamiento:** lleva el siguiente símbolo de entrada a la cima de la pila
- **Reducción:** extrae tantos símbolos de la pila como símbolos tiene la parte derecha de la regla de sustitución, por la cual se quiere reducir, y los sustituye por el símbolo no terminal de la parte izquierda de la regla.

3.4.1 Gramáticas LR(k)

En análisis ascendente por desplazamiento-reducción se utilizan gramáticas $LR(k)$:

- $L(\text{left})$: la entrada se lee de **izquierda a derecha**

- R (right): se construyen **derivaciones** por la **derecha** en orden inverso
- (k) : se leen k **componentes** léxicos por **anticipado**. Se pueden demostrar que, para cada gramática $LR(k)$ con $k > 1$, hay una gramática $LR(1)$ equivalente. Por esto, normalmente se trabaja con $k = 1$.

Las gramáticas $LR(1)$ son **no ambiguas** y permiten describir más lenguajes que las gramáticas $LL(1)$, **pues las engloban**. Hay tres tipos de gramáticas LR con el mismo esquema de control, basado en la construcción y uso de una tabla de análisis. Por la **manera de construir** esa **tabla** distinguimos:

- **Gramáticas $SLR(1)$** (simple LR): presentan la construcción más sencilla (usaremos esta).
- **Gramáticas $LR(1)$** : permiten describir un rango mayor de gramáticas, pero la construcción es más compleja.
- **Gramáticas $LALR(1)$** (look ahead LR): suponen un compromiso entre las dos anteriores

3.4.2 Análisis

Utilizaremos un procedimiento basado en el diseño de un **AFD**, que nos dirá si una **entrada pertenece al lenguaje** generado por la gramática y cuyos **estados** son **conjuntos de elementos** que representan **situaciones equivalentes en el análisis de una entrada**.

Una función a la que llamaremos `ir_a` nos **permitirá simular las transiciones** del autómata para cada uno de los símbolos de la gramática. Para una gramática G creamos su **gramática aumentada**, resultado de añadir artificialmente la regla $S' \rightarrow S\$$. Cuando volvamos a reducir mediante esta nueva regla **aceptamos**.

Elementos

Un **elemento** $LR(0)$ es una **regla** simple que contiene en algún lugar de la parte derecha una **marca** que separa los **símbolos** de la regla que han sido **reconocidos** de los **pendientes de reconocer**.

Para las reglas de sustitución $A \rightarrow \epsilon$ sólo se obtiene el elemento $A \rightarrow \cdot$.

Clausura

La función `clausura()` se aplica a un **conjunto de elementos** I y devuelve **un conjunto de elementos** que son **equivalentes** respecto a **dónde llegó el análisis**. El conjunto de elementos que devuelve $clausura(I)$ se construye de la siguiente manera:

1. Todo elemento de I se añade a $clausura(I)$
2. Si $A \rightarrow \alpha \cdot B\beta$ está en $clausura(I)$ y $B \rightarrow \gamma$ es una regla de la gramática, entonces añadir $B \rightarrow \cdot\gamma$ a $clausura(I)$
3. Aplicar el paso 2 hasta que no se puedan añadir nuevo elementos a la $clausura(I)$

Si $A \rightarrow \alpha \cdot B\beta$ está en $clausura(I)$, acabamos de ver la entrada de una cadena derivable de α , y esperamos encontrar a continuación una cadena derivable de $B\beta$, es decir, una de las cadenas resultado de sustituir B de todas las formas posibles, por lo que añadimos $B \rightarrow \cdot y$. Si y comienza por un **no terminal** repetimos. Terminamos cuando tenemos todos los elementos que indiquen el principio de una cadena esperada a partir del momento actual.

Traducción: La **clausura** se usa cuando después del punto hay un **no terminal**.

$A \rightarrow \cdot B - D$

Como después del punto está B , el parser se pregunta: ¿Cómo puede empezar B ? Entonces añade todas las reglas de B con el punto al principio. Si tenemos:

$B \rightarrow x$
 $B \rightarrow y z$

añadiríamos a la clausura:

$B \rightarrow \cdot x$
 $B \rightarrow \cdot y z$

$ir_a(I, X)$

Desde el conjunto de elementos I , si reconozco el símbolo X , ¿a qué nuevo conjunto de elementos paso?

La función $ir_a(I, X)$ se aplica a un **conjunto de elementos** I y a un **símbolo de la gramática** $X \in (V \cup \Sigma)$ y devuelve un **nuevo conjunto de elementos**.

$ir_a(I, X)$ se define como la **clausura** del conjunto de **todos los elementos** $A \rightarrow \alpha X \cdot \beta$ tales que $A \rightarrow \alpha \cdot \beta$ está en I , es decir, aquellos elementos que después de I dan por reconocido el símbolo X . Intuitivamente, la función $ir_a(I, X)$ adelanta el símbolo X en el análisis sintáctico ascendente.

Colección Canónica LR(0)

Llamos **colección canónica** $LR(0)$ al conjunto de todos los **posibles conjuntos de elementos de una gramática**. Para generar la colección canónica de una gramática haremos:

1. Crear la gramática aumentada, resultado de añadir artificialmente la regla $S' \rightarrow S\$$
2. El primer conjunto de elementos es $I_0 = clausura(S' \rightarrow S\$)$

3. Calcular todos los posibles conjuntos resultado de aplicar $ir_a(I_0, X)$ para todos los símbolos $X \in (V \cup \Sigma)$ y se descartan los conjuntos vacíos.
4. Aplicamos ir_a de nuevo a $I_1, I_2, \dots$ hasta no obtener nuevos conjuntos.

Si consideramos cada conjunto I_j como un estado de un autómata finito, ir_a nos proporciona su **tabla de transiciones**.

Tabla de acciones

Podemos construir una **tabla de acciones** que permitirá al analizador sintáctico ascendente **decidir entre desplazar y reducir**. El procedimiento es el siguiente:

1. Construir la colección canónica $LR(0) = I_0, I_1, \dots, I_n$
2. Para los elementos I_j de la forma $A \rightarrow \alpha \cdot a\beta$, donde $ir_a(I_j, a) = I_k$, entonces entonces acción $[j, a] = \text{desplazar}$ la entrada a e ir al estado I_k
3. Para los elementos de I_j , de la forma $A \rightarrow \alpha \cdot$, entonces $acción[j, s] = \text{reducir } A \rightarrow \alpha$ para cualquier $s \in SIGUIENTES(A)$
4. Si el elemento $S' \rightarrow S \cdot \$$ está en I_j , entonces $acción[j, \$] = \text{aceptar}$

Las **entradas vacías** de la tabla de acciones representan estados de **error**.

Procedimiento de análisis

Podemos proporcionar un procedimiento de análisis basado en el uso de una **tabla de control** para seleccionar la **siguiente regla** de sustitución y una **pila** para almacenar los **símbolos** gramaticales.

El algoritmo es el siguiente:

1. Inicializar la pila con el estado 0
2. Para cada estado de la pila i y símbolo de entrada a consultamos la celda $acción[i, a]$.
 - Si $acción[i, a] = dn$ entonces **desplazamos** la entrada, vamos al estado n y lo introducimos en la cima de la pila
 - Si $acción[i, a] = rn$ entonces **reducimos** mediante la regla (n) , de la forma $A \rightarrow \beta$; para ello si m es la longitud de β , eliminamos m estados de la cima de la pila y a continuación vamos al estado $ir_a(h, A)$, donde h es el estado que ha quedado en la pila
 - Si $acción[i, a] = \text{aceptar}$, **completamos el análisis**
 - Si $acción[i, a] = \emptyset$, error e invocamos la **gestión de errores**

Una gramática es $SLR(1)$ si en la **tabla de análisis** aparece como máximo una entrada en cada **celda**. Cuando no es así se pueden señalar dos tipos de conflictos:

- **Desplazamiento-reducción:** sucede cuando en la **misma celda** aparece una acción de **desplazar** y una de **reducir**. Normalmente para poder continuar el análisis se elige una **acción**

por defecto, generalmente la de **desplazar**

- **Reducción-reducción:** sucede cuando en la **misma celda**, la entrada se puede **reducir por dos reglas distintas**. La solución más razonable es **modificar la gramática** para que esto no ocurra.

3.4.4 Gestión de Errores en ASA

Los **errores sintácticos** suceden cuando a partir del **estado actual de la cima** de la pila **no** hay ninguna **transición definida** para el **símbolo actual** de la entrada. En esta situación el **estado** de la pila representa el **contexto a la izquierda dle error**, y el **resto de la cadena** de entrada, el **contexto a la derecha**.

La **recuperación del error** se lleva a cabo **modificando la pila** y/o **cadena de entrada**, hasta que el estado y la cadena le permiten al analizador **continuar** el análisis. Mostraremos dos métodos de gestión de errores:

- **Métodos heurísticos:** suponen la programación de **rutinas específicas** para **cada celda** en la que se produce un error
- **Métodos de análisis de transiciones:**
 1. Explorar la pila hacia abajo hasta encontrar un estado d con un ir_a para un no terminal A
 2. Descartar cero o más símbolos en la entrada hasta encontrar uno que pueda seguir A de acuerdo con la gramática
 3. Mete el estado $ir_a(d, A)$ en la pila y continuar

4. Análisis Semántico

Podemos entender la **semántica** de un lenguaje de programación como el conjunto de **reglas** que especifican el **significado** de las **sentencias sintácticamente correctas**.

El análisis semántico se puede llevar a cabo como una **tarea** de análisis **independiente** de las demás o bien **paralelo al análisis sintáctico**.

Como **salida** del análisis semántico se genera automáticamente una **representación en forma de código intermedio** que pasa a las **fases de síntesis de la compilación**.

4.1 Análisis Semántico

Cada **símbolo** de la gramática adquiere un determinado **significado** según el **contexto de aparición en el código fuente**. Para que los símbolos puedan tener significado, es necesario **aumentar la gramática** añadiendo un conjunto de **atributos** a los símbolos y unas **acciones semánticas** a las reglas sintácticas.

La especificación de la semántica se puede realizar mediante:

- **Lenguajes especiales de especificación:** como VMD (para PL/1), Anna (para Ada), etc.
- **Gramáticas de atributos:** llamaremos **atributo** de un símbolo de la gramática a todo ítem de **información** añadido al **árbol de derivación** durante el **análisis semántico**.

4.1.1 Gramáticas de Atributos

Una **gramática de atributos** es toda aquella **gramática independiente del contexto** a la que se añade un **sistema de atributos**.

Un **sistema de atributos** está formado por:

- Un conjunto de **atributos semánticos:** un atributo de un **símbolo** X es una variable que representa una determinada **característica de X** que puede tomar un **valor de entre un conjunto, finito o no**, de valores posibles. Denotaremos el atributo a del símbolo X como $X.a$
- Un conjunto de **acciones semánticas:** una acción semántica es un **algoritmo** asociado a una **regla de la gramática** cuyo objetivo es **calcular el valor de alguno de los atributos** de los **símbolos de dicha regla**.

4.1.2 Propagación de Atributos

Llamamos **propagación** al cálculo del **valor** de un **atributo** en **función del valor de otros**. La propagación se realiza siguiendo el **árbol de derivación** que devuelve el análisis **sintáctico**, e incluso se puede desarrollar **al mismo tiempo**.

Según los símbolos que intervienen en una acción semántica, para calcular el valor del atributos de un símbolo, hablamos de dos tipos de atributos:

- Atributos **sintetizados**: son atributos asociados a **no terminales** de la **parte izquierda** de una regla de sustitución
 - Se calculan a partir de los **nodos hijos del subárbol** en que aparecen
 - La información recorre el árbol verticalmente
- Atributos **heredados**: son atributos asociados a símbolos de la **parte derecha** de una sustitución.
 - Se calculan a partir de los **atributos del nodo padre** y de los **nodos hermanos** dentro de un mismo subárbol
 - La información recorre el árbol **horizontalmente**

4.1.3 Grafo de Dependencias

La **propagación** se ha de realizar siguiendo el **orden** implícito en la **dependencia** que existe entre los atributos, **no** podemos **evaluar** una regla semántica **antes** de disponer de **todos los atributos involucrados**.

Este orden se representa mediante un **grado de dependencias acíclico y dirigido**, para cuya construcción representamos las dependencias entre atributos de la forma $b = f(c_1, c_2, \dots, c_n)$. El grado tendrá un **nodo** para cada **atributo** de cada regla semántica, y un **arco de** c_i a b si b depende de c_i :

1. Se evalúan los nodos a los que no llega ningún arco y se marcan como calculados
2. Se evalúan los nodos en los que todos los arcos proceden de nodos marcados y se marcan como calculados
3. Continuamos hasta que no quedan más nodos por evaluar

4.1.4 Verificación de Tipos

La **verificación de tipos** asegura que el tipo de **cada** una de las **construcciones** del código fuente **coincide** con lo **previsto** en el **diseño** de la gramática.

Un **sistema de tipos** es una serie de **reglas** que permiten **asignar tipos** a las distintas **construcciones** del código fuente:

1. En primer lugar, a los **tipos básicos**
2. A continuación, a expresiones de tipos para **matrices, estructuras, punteros**, etc

Estas reglas se **incluyen** en el **árbol de análisis sintáctico**, de un modo similar a las de asignación y propagación de atributos.

Según la verificación de tipos se distingue entre lenguajes fuerte o débilmente tipificados y lenguajes con verificación estática o dinámica de tipos.

- Los lenguajes **fuertemente tipificados** serían aquellos en los que **no** es posible un **error de tipo** en tiempo de **ejecución** (Java)
- Los lenguajes **débilmente tipificados** serían aquellos en los que es posible usar un **valor de un tipo como si fuese de otro tipo** (C)
- Los lenguajes con **tipificación estática** son aquellos que realizan la **verificación de los tipos en tiempo de compilación** (Java)
- Los lenguajes con **tipificación dinámica** son aquellos que realizan la **verificación de tipos en tiempo de ejecución** (Python)

4.2 Generación de Código Intermedio

Una **representación intermedia** es una **estructura de datos** que **representa** al **código fuente** durante el proceso de **traducción a código objeto**. El **árbol de análisis sintáctico** y la **TS** son representaciones del código fuente, pero se parecen muy **poco** al **código ejecutable**. La **notación postfixa** también es una representación intermedia utilizada a menudo en la traducción de expresiones.

4.2.1 Código de Tres Direcciones

Se define como **una secuencia de instrucciones** de la forma $x = y \text{ op } z$ donde op representa a cualquier operador y x y z representan **símbolos** definidos por el **programador** (residentes en la TS), **variables temporales** generadas por el compilador, **constantes**.

Existen **muchos tipos** de códigos de tres direcciones, pero todos se desarrollan para algún tipo de **máquina virtual**.

Es una **linealización** del **árbol de análisis** sintáctico donde se asignan **nombres temporales** a los **nodos internos**. El proceso de **traducción** del código de tres direcciones a **código máquina** suele ser bastante **sencillo**.

Para poder representar **construcciones de un lenguaje de alto nivel**, es necesario introducir en el código intermedio:

- **Instrucciones de asignación con operador binario**, donde op es una operación aritmética o lógica: $X := y \text{ op } z$
- **Instrucciones de asignación con operador unitario**, donde op es una operación de opuesto, negación, desplazamiento o conversión de tipo: $x := op \ y$
- **Instrucciones de copia**: $x := y$
- **El salto condicional**, donde se utiliza una etiqueta E para indicar la siguiente instrucción a ejecutar: *goto* E
- **El salto condicional**, donde se ejecuta la instrucción E o la siguiente en la secuencia en función del **resultado de un operador de relación**: *if* $x \text{ oprel } y$ *goto* E
- **Llamadas a procedimientos**, de la forma $p(x_1, x_2, \dots, x_n)$:
 - *param* x_1
 - *param* x_n
 - *call* p, n
 - y devoluciones mediante la instrucción *return* y
- **Asignaciones con índices**:
 - $x := y[i]$
 - $x[i] := y'$
- **Asignaciones de direcciones y punteros**:
 - $x := y$
 - $x := *y$
 - $*x := y$

Un **conjunto de operadores pequeño** es más **fácil de implementar** en una máquina objeto. Sin embargo, si resulta ser un **conjunto limitado** puede obligar a generar **largas secuencias** de instrucciones para algunas construcciones del lenguaje fuente.

4.2.2 Implementaciones

El código intermedio suele **almacenarse** como una **lista enlazada de registros**, donde cada registro representa una instrucción. Existen dos implementaciones:

- Un **cuádruple** es una estructura de tipo registro con **cuatro campos**: `op`, `result`, `arg1` y `arg2`. Las instrucciones que no usan todos los campos dejan vacíos los que no se utilizan. La instrucción $x = y + z$ se representa como `( suma, x, y, z )`
- Un **triple** es una simplificación en la que **cada instrucción** representa a una **variable temporal**, utilizando así sólo **tres campos**: `op`, `arg1`, `arg2`. La instrucción $x = y + z$ se representa como `01(suma, y, z)` y `02(asigna, x, (01))`. Está en desventaja respecto a los cuádruples cuando la **optimización** de código **supone** su **reorganización** ya que si **modificamos sus posiciones** tendremos que **modificar sus referencias**.

- Para evitar este problema se introduce los **triples indirectos**: una lista de **punteros a triples** que evita modificar las posiciones de los triples. Cualquier optimización de código se realiza sobre esta lista.

5. Generación y Optimización de Código

![[Nota]] Estos apuntes no están muy bien, sirven para responder los ejercicios típicos que pone el rivera en el examen pero no desglosan los conceptos para poder entender todo bien. Es que estaba quemado ya de currar este cuatri y me dio excesiva pereza la parte del rivera.

5.1 Bloques básicos

Un **bloque básico** es una secuencia de instrucciones que se ejecuta seguida.

Cumple:

1. Solo se entra por la primera instrucción.
2. Solo se sale por la última.
3. No hay saltos a instrucciones intermedias.

Para encontrarlos, buscas **líderes**:

1. La primera instrucción es líder.
2. Todo destino de salto es líder.
3. La instrucción justo después de un salto también es líder.

Ejemplo:

```
1: b = 0
2: a = c + d
3: b = a + e
4: d = a + b
5: p = d
6: b = c + d
7: if (b == 0) goto 13
8: if (b > 0) goto 2
9: a = b + c
10: *r = a
11: b = d + e
12: goto 1
```

```
13: *r = *r + a
14: return
```

Líderes:

```
1 → primera instrucción y destino de goto 1
2 → destino de goto 2
8 → instrucción después del salto de la línea 7
9 → instrucción después del salto de la línea 8
13 → destino del goto 13
```

Bloques:

```
B1 = {1}
B2 = {2,3,4,5,6,7}
B3 = {8}
B4 = {9,10,11,12}
B5 = {13,14}
```

5.2 Variables vivas y siguiente uso

Una variable está **viva** si su valor se va a usar más adelante.

El **siguiente uso** es la próxima línea donde se usará.

Ejemplo:

```
1: t = a + b
3: x = t * c
```

Después de la línea 1:

```
t está viva
siguiente uso de t = 3
```

Tu profesor usa una tabla con dos datos por variable:

```
v = viva
nv = no viva
```

número = siguiente uso
0 = no hay siguiente uso dentro del bloque

Cómo se hace la tabla

Para una instrucción:

```
x = y op z
```

vas **de abajo hacia arriba**.

En cada línea haces:

1. Apuntas el estado actual.
2. Matas x:
x = nv, 0
3. Marcas y y z:
y = v, línea actual
z = v, línea actual

Para una condición:

```
if (b == 0) goto 13
```

solo se usa **b**, así que:

```
b = v, línea actual
```

Ejemplo

Bloque:

```
2: a = c + d  
3: b = a + e  
4: d = a + b  
5: p = d  
6: b = c + d  
7: if (b == 0) goto 13
```

Si no te dan más información global, tus apuntes dicen que se suele asumir:

Variables no temporales vivas al salir.
Temporales no vivas al salir.

Aquí todas parecen variables normales, así que puedes empezar suponiendo vivas al salir:

```
a b c d e p  
v v v v v v  
0 0 0 0 0 0
```

Ahora recorres desde abajo.

Línea 7:

```
if (b == 0) goto 13
```

Usa `b`.

```
b = v, 7
```

Línea 6:

```
b = c + d
```

Antes de modificar, `b` estaba viva con uso 7.

Luego matas `b`, y marcas `c` y `d` con uso 6.

Línea 5:

```
p = d
```

Matas `p`, marcas `d` con uso 5.

Línea 4:

```
d = a + b
```

Matas `d`, marcas `a` y `b` con uso 4.

Y así sucesivamente.

El formato final sería parecido al de tu profesor:

Salida:

a	b	c	d	e	p
v	v	v	v	v	v
0	0	0	0	0	0

7:

a	b	c	d	e	p
v	v	v	v	v	v
0	7	0	0	0	0

6:

a	b	c	d	e	p
v	nv	v	v	v	v
0	0	6	6	0	0

5:

a	b	c	d	e	p
v	nv	v	v	v	nv
0	0	6	5	0	0

No hace falta memorizar la tabla: memoriza la regla de **matar la definida y marcar los operandos**.

5.3 DAG / GDA de un bloque básico

Un **DAG** o **GDA** es un grafo que representa los cálculos de un bloque.

Sirve para:

- detectar subexpresiones comunes
- eliminar código muerto
- ver dependencias entre instrucciones
- reordenar instrucciones si se puede

Cómo se construye:

1. Creas hojas para los valores iniciales de variables.
2. Cada operación crea un nodo.
3. Los hijos del nodo son las últimas definiciones de sus operandos.
4. El nodo se etiqueta con la variable que acaba de definirse.

Ejemplo:

```
1: a = b + c
2: d = a - e
3: x = b + c
```

DAG:

```

b0    c0
 \    /
  +    etiqueta: a, x
  |
a/x

a/x    e0
 \    /
  -    etiqueta: d
```

La línea 3 no crea un nodo nuevo porque `b + c` ya estaba calculado. Se reutiliza el mismo nodo.

5.4 DAG con punteros

Ejemplo:

```
1: u = a + b
2: t = b + c
3: *p = t
4: a = a * b
5: t = a + c
6: a = 2 * t
7: a = a / u
```

Caso A: el compilador sabe que `p` no apunta a ninguna variable usada

Entonces:

```
*p = t
```

es una escritura indirecta, pero **no invalida** los nodos de `a`, `b`, `c`, `u`, `t`, etc.

El DAG se puede seguir construyendo normalmente.

Textualmente:

```
n1 = a0 + b0      etiqueta: u
n2 = b0 + c0      etiqueta: t

n3 = *(p0, n2)     efecto lateral: *p = t

n4 = a0 * b0      etiqueta: a
n5 = n4 + c0      etiqueta: t
n6 = 2 * n5       etiqueta: a
n7 = n6 / n1      etiqueta: a
```

Lo importante es:

La línea `*p = t` no destruye el DAG anterior porque sabemos que `p` no toca esas variables.

Caso B: el compilador NO sabe a qué apunta `p`

Entonces `*p = t` puede modificar cualquier variable.

Por tanto:

```
*p = t
```

se considera una **barrera**.

En tus apuntes aparece esta idea: una asignación indirecta `*=`, si no se sabe a qué puede apuntar, elimina los nodos construidos hasta ahora.

Entonces el DAG se corta en dos partes.

Primera parte:

```
n1 = a0 + b0      etiqueta: u
n2 = b0 + c0      etiqueta: t
n3 = *(p0, n2)
```

Después de `*p = t`, empiezas casi desde cero:

```
n4 = a1 * b1      etiqueta: a
n5 = n4 + c1      etiqueta: t
```



```
n6 = 2 * n5      etiqueta: a
n7 = n6 / u1     etiqueta: a
```

Uso `a1`, `b1`, `c1`, `u1` para indicar “valor nuevo/desconocido después de la escritura indirecta”.

Si no sabemos a qué apunta `p`, la instrucción `*p = t` puede modificar cualquier variable accesible, por tanto invalida los nodos anteriores del DAG.

5.5. DAG con arrays

Ejemplo:

```
1: a = b + c
2: b = a - d
3: d = x[k]
4: x[j] = b
5: c = x[k]
6: c = c + d
7: e = b / c
```

Regla principal:

Una escritura en un array puede cambiar lecturas posteriores del mismo array.

La línea 3 lee:

```
d = x[k]
```

La línea 4 escribe:

```
x[j] = b
```

La línea 5 vuelve a leer:

```
c = x[k]
```

¿Podemos reutilizar la lectura de la línea 3 para la línea 5?

No, salvo que sepamos que:

```
j != k
```

Porque si `j = k`, entonces la línea 4 modifica justo `x[k]`.

Por tanto:

```
d = x[k]
x[j] = b
c = x[k]
```

normalmente genera dos lecturas distintas de `x[k]`.

Representación:

```
d = x[k]      → nodo = [](x, k), etiqueta d
x[j] = b      → nodo []=(x, j, b), modifica x
c = x[k]      → nuevo nodo = [](x, k), etiqueta c
```

5.6 Descriptores de registros y variables

Esto se usa con `obtenReg`.

Un **descriptor de registro** dice qué variables están actualmente en cada registro.

Ejemplo:

```
R1: d
R2: a, t
R3: u, v
```

Significa:

```
R1 contiene el valor actual de d
R2 contiene el valor actual de a y t
R3 contiene el valor actual de u y v
```

Un **descriptor de variable** dice dónde está el valor actual de cada variable.

Ejemplo:

a: R2
d: R1
t: R2
u: R3
v: R3

5.7 Ejercicio típico: traducir **b = v**

Situación:

R1: d
R2: a, t
R3: u, v

Queremos traducir:

b = v

Como **v** ya está en **R3**, no hace falta generar ningún **LD**.

Y como es una copia, no hace falta instrucción máquina. Solo actualizamos descriptores:

Antes:

R3: u, v

Después:

R3: u, v, b

Y el descriptor de **b** pasa a ser:

b: R3

Respuesta de examen:

No se genera ninguna instrucción.
Se actualizan descriptores:

```
R3 = {u, v, b}
b = {R3}
```

Si al final del bloque `b` está viva y no está en memoria, entonces más tarde habría que hacer:

```
ST b, R3
```

Pero para la operación `b = v`, no se emite código.

5.8 Qué usa `obtenReg`

¿Qué usa la función `obtenReg` para decidir?

Respuesta:

- Los descriptores de registros.
- Los descriptores de direcciones o variables.
- La información de vida.
- La información de siguiente uso.
- La instrucción concreta que se está traduciendo.

La idea es escoger registros intentando evitar `ST` innecesarios.

Si una variable de un registro:

- ya está guardada en memoria,
- no se va a usar más,
- o va a ser sobrescrita,

entonces puedo reutilizar ese registro sin almacenar.

Si no, tengo que hacer un **derrame**:

```
ST variable, R
```

5.9 Coste de `LD R1, *8(R2)`

Criterio 1: coste Aho / longitud de instrucción

Regla:

`coste = 1 + coste de los modos de direccionamiento`

- Registro `R1`: coste adicional 0.
- Dirección indirecta con desplazamiento `*8(R2)`: suele contar desplazamiento `8` y acceso indirecto.

Resultado típico:

`Coste = 3`

Criterio 2: accesos a memoria

Aquí miras cuántos accesos a memoria provoca.

Para:

`LD R1, *8(R2)`

se accede a una dirección calculada a partir de `8 + contenido(R2)` y luego se obtiene el contenido indirecto.

En clase normalmente se razona también como:

`Coste = 3`

5.10 Reglas de reensamblado

El **reensamblado** es volver del DAG a código de tres direcciones.

Reglas importantes:

1. Un nodo debe generarse después de sus hijos.
2. Una escritura en array debe respetar el orden respecto a lecturas/escrituras anteriores del mismo array.

3. Una lectura de array debe ir después de escrituras anteriores que puedan afectarla.
4. Una instrucción no debe cruzar una escritura por puntero si puede cambiar su valor.
5. Si un nodo tiene varias variables vivas adjuntas, hay que generar copias.

Ejemplo:

```
a = b + c  
d = b + c
```

DAG:

```
n1 = b + c, etiquetas {a, d}
```

Si `a` y `d` están vivas al salir, al reensamblar necesitas:

```
a = b + c  
d = a
```

o al revés.

Porque ambas variables necesitan conservar ese valor.